

Informe Actividad 1

Laberinto con Grafos

Integrantes:

Michael Stevens Cardenas Urbano (2610304)
Luis Santiago Arenas Hincapié (2435300)
Brigitte Vanesa Chavez Riascos(2510107)

Asignatura:

Inteligencia Artificial

Docente:

Andres Mauricio Valencia Restrepo

Facultad de Ingeniería

Escuela de Ingeniería de Sistemas y Computación
Ingeniería de Sistemas

Santiago de Cali, 12 de Septiembre de 2026

1. Modelado del ambiente: grafo

El ambiente del programa es un laberinto representado inicialmente como una matriz bidimensional de números enteros. Cada valor de la matriz indica el tipo de celda: camino libre (0), muro (1), penalización (2), premio (3) y meta (9). A partir de esta matriz, el programa construye automáticamente un grafo no dirigido utilizando la clase `nx.Graph()` de NetworkX.

Cada celda de la matriz cuyo valor sea distinto de 1 se convierte en un nodo del grafo, identificado mediante la tupla `(fila, columna)` correspondiente a su posición. Las celdas marcadas con el valor 1 (muros) no generan ningún nodo, por lo que quedan excluidas del grafo y, en consecuencia, no pueden formar parte de ningún desplazamiento.

Las aristas del grafo representan la posibilidad de movimiento directo entre dos posiciones adyacentes. Se crean únicamente entre celdas vecinas en sentido vertical u horizontal (arriba, abajo, izquierda o derecha), sin permitir conexiones diagonales, y solo si ambas celdas son transitables (distintas de 1).

De esta forma, la existencia de un camino libre entre dos posiciones se traduce directamente en la existencia de una arista en el grafo. Para comprobar si el jugador puede moverse de una posición a otra, el programa consulta al grafo mediante `G.has_edge(pos, destino)`: si la arista existe, el movimiento es válido; si no existe, se interpreta como la presencia de un muro.

Los distintos tipos de casillas (penalización, premio, meta) no afectan la estructura del grafo ni la existencia de las aristas; su efecto se aplica únicamente sobre el estado del jugador (monedas) en el momento en que este ocupa dicha celda, y se identifican consultando el valor original de la matriz en la posición actual.

NetworkX cumple así dos funciones principales dentro del programa: primero, permite construir y almacenar la estructura del laberinto como un grafo mediante `add_node` y `add_edge`; segundo, permite validar en tiempo real, sin necesidad de algoritmos de búsqueda de caminos, si un movimiento solicitado por el usuario es físicamente posible dentro del laberinto.

2. Explicación del código

2.1. Representación de la matriz y construcción del grafo

El laberinto se define como una lista de listas de Python (`laberinto`). A partir de sus dimensiones, se recorre la matriz dos veces: una primera vez para crear los nodos y una segunda para crear las aristas.

```
for f in range(filas):
    for c in range(columnas):
        if laberinto[f][c] != 1:
            G.add_node((f, c), valor=laberinto[f][c])
```

Para las aristas, cada celda transitable revisa únicamente a su vecino de abajo y a su vecino de la derecha, evitando así duplicar conexiones ya creadas en iteraciones anteriores.

```

if f + 1 < filas and laberinto[f + 1][c] != 1:
    G.add_edge(actual, (f + 1, c))
if c + 1 < columnas and laberinto[f][c + 1] != 1:
    G.add_edge(actual, (f, c + 1))

```

2.2. Posición inicial y sistema de movimiento

El jugador inicia en la posición (0, 0) con 30 monedas (`coins`). El movimiento se controla mediante las teclas `w`, `a`, `s` y `d`, asociadas a desplazamientos verticales y horizontales a través de un diccionario de direcciones:

```

direcciones = {
    "w": (-1, 0),
    "s": (1, 0),
    "a": (0, -1),
    "d": (0, 1)
}

```

A partir de la posición actual y la dirección ingresada, se calcula la celda destino sumando ambos valores. En ningún momento el programa decide automáticamente hacia dónde moverse; toda decisión de movimiento proviene del usuario.

2.3. Comprobación de movimientos válidos

Antes de mover al jugador, el programa verifica que el destino exista como nodo del grafo y que exista una arista entre la posición actual y dicho destino:

```

if G.has_edge(pos, destino):
    pos = destino
    valor = procesar_celda(pos)
else:
    print("hay un muro")

```

Si la arista no existe, se informa al usuario que hay un muro y la posición del jugador no cambia.

2.4. Actualización de monedas y condiciones especiales

Una vez confirmado el movimiento, la función `procesar_celda` revisa el valor original de la matriz en la nueva posición para aplicar su efecto: una penalización resta 5 monedas, un premio suma 10 monedas, y la meta finaliza la partida de forma exitosa. Si el contador de monedas llega a cero o menos, la partida termina sin haber alcanzado la meta.

2.5. Visualización del laberinto

El grafo se representa gráficamente mediante `matplotlib` y la función `nx.draw()`, utilizando las mismas coordenadas (`fila`, `columna`) de cada nodo como posición en el plano. Los nodos se colorean según el tipo de celda (camino, penalización, premio, meta) y la posición actual del jugador se resalta con un color distinto. El dibujo se actualiza automáticamente después de cada movimiento válido, y también puede solicitarse manualmente mediante el comando `ver`.

3. Evidencias de funcionamiento

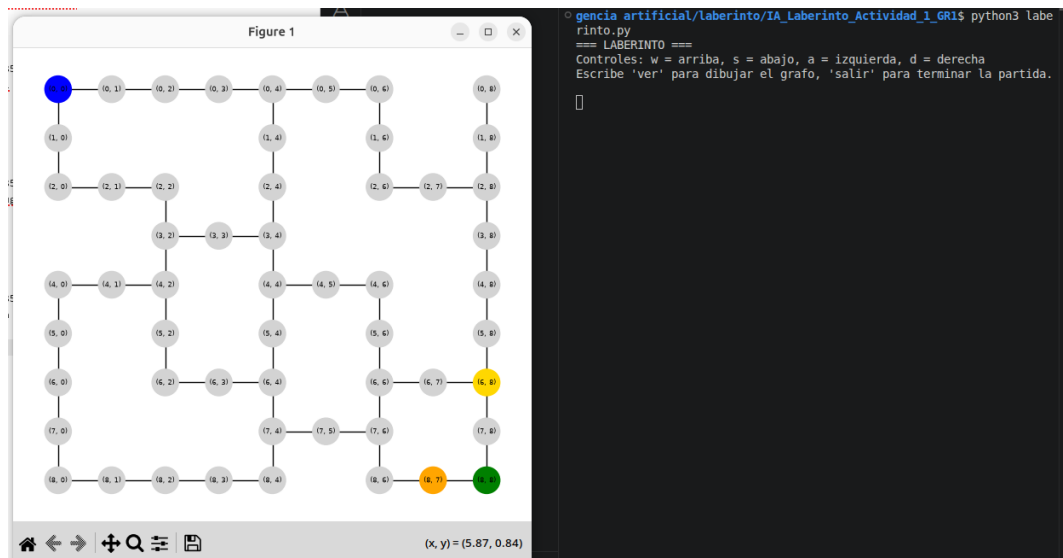


Figura 1: Ejecución inicial del programa.

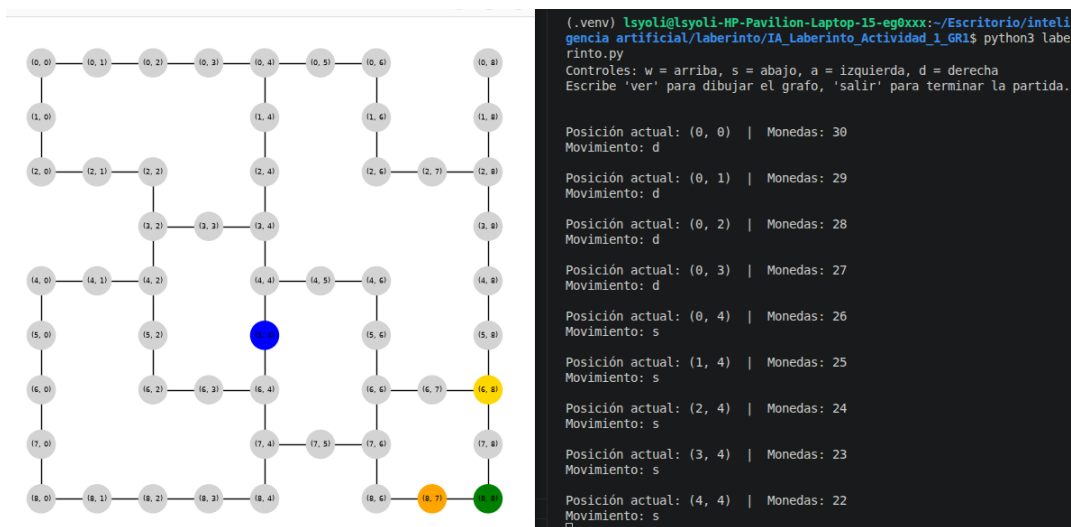


Figura 2: Movimiento del jugador dentro del laberinto.

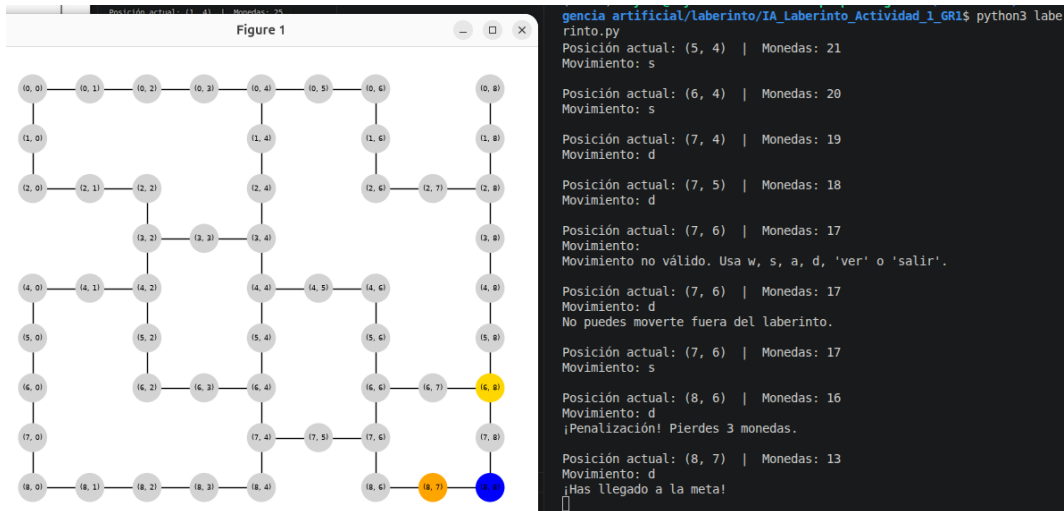


Figura 3: Llegada a la meta / resultado final.